MRP - Robust CNN-Based Binary Image Recognition System

By Marina Rizk

MSCS2201

Motivation, Main idea, and Results

- Motivation:
  - Explore how Convolutional Neural Networks (CNNs) can automatically classify images, using cat vs dog classification as a practical example. This task highlights key computer vision challenges like feature extraction and generalization, while providing a clear way to evaluate performance.
- Main Idea:
  - This project uses a Convolutional Neural Network (CNN) to automatically learn visual features and classify images of cats and dogs. A separate inference script loads the trained model to predict new images with confidence scores.
- Results:
  - The model trained for 12 epochs and achieved ~97% accuracy on validation data, with high precision and recall for both classes.

Literature Review

- Deep learning dominance: CNNs have completely replaced traditional methods by automatically learning hierarchical features directly from raw pixels, eliminating manual feature engineering.
- Transfer learning standard: Pre-trained models (like MobileNetV2) consistently outperform scratch-trained networks by leveraging features learned from large datasets like ImageNet, drastically reducing training time and data requirements.
- MobileNetV2: is a lightweight CNN built for speed and efficiency on phones or low-power devices. It uses depthwise separable convolutions that splits the filtering process into two simpler steps, drastically reducing computation while keeping good accuracy.
- Trade-offs: Bigger models = more accurate but slower. Smaller models like MobileNetV2 = faster and use less battery, perfect for phones. Pick whatever is more efficient for the project.

Approach Used

- Dataset:
  - https://download.microsoft.com/download/3/E/1/3E1C3F21-ECDB-4869-8368-6DEBA77B919F/kagglecatsanddogs_5340.zip
- Data Preparation
  - Custom cleaning function using PIL to verify and remove corrupted images
  - image_dataset_from_directory with 80/20 validation split
- Model Architecture
  - Base: MobileNetV2 (pre-trained on ImageNet, frozen)
    - Contains ~53 convolutional layers (pre-built)
    - https://www.kaggle.com/models/google/mobilenet-v2/tensorFlow2/tf2-preview-classification/4?tfhub-redirect=true
  - Data augmentation: RandomFlip + RandomRotation

Approach Used

- Training
  - Transfer learning with frozen base model (non-trainable layers in MobileNetV2)
  - Loss: Binary crossentropy
  - Epochs: 12, Batch size: 32
- Total Parameters: MobileNetV2: ~2,250,000, Dense(64): 81,984, Dense(1): 65
  - Total ≈ 2.33 *million*
- Inference (test.py)
  - Loads trained model from .h5 file
  - Preprocesses single image: PIL load → RGB convert → resize (160×160)
  - Returns class (Cat/Dog) with confidence percentage

Code Used

- Pretrained CNN Backbone (MobileNetV2) ~53 convolutional layers used for feature extraction (ImageNet pretrained)

```
base_model = tf.keras.applications.MobileNetV2(
```

- CNN Layers Frozen (Transfer Learning) ~2.25M parameters are **non-trainable**, reducing computation and overfitting

```
base_model.trainable = False
```

- Custom Classification Layers (Trainable) - Only these layers learn → convert features into cat vs dog prediction

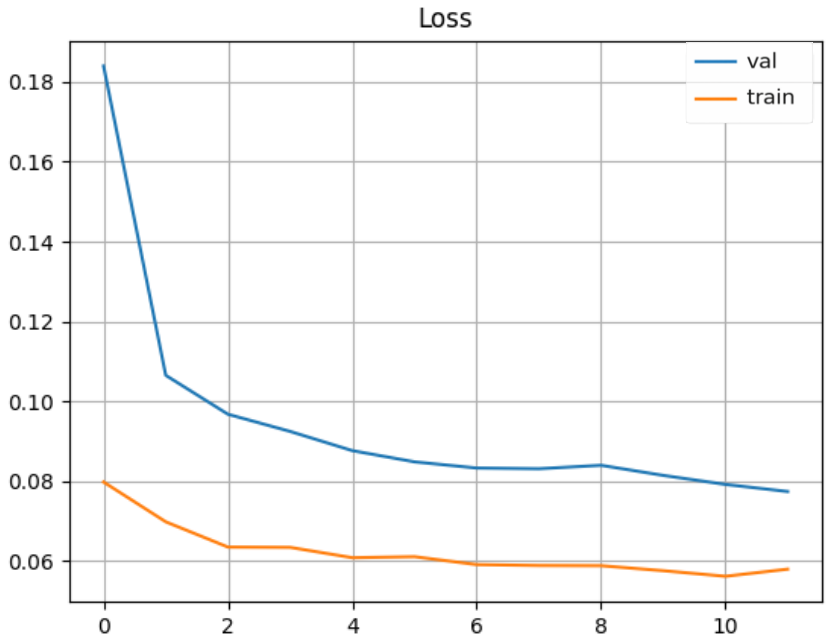
```
tf.keras.layers.GlobalAveragePooling2D(),
tf.keras.layers.Dense( units: 64, activation='relu'),
tf.keras.layers.Dense( units: 1, activation='sigmoid')
```

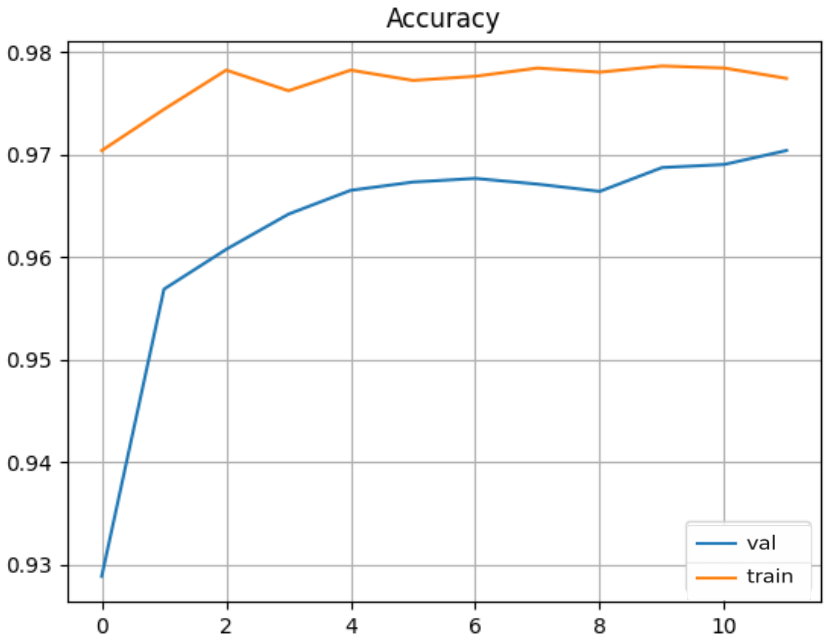
- **Total Parameters** ~2.3M total parameters - Majority from CNN (frozen) - Few thousand are trainable (Dense layers)
- Data Processing Layers - Normalize input and augment data to improve generalization

```
data_aug = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.1),
])
```

- **Training Setup - Uses Adam optimizer, binary cross-entropy, trained for 12 epochs**

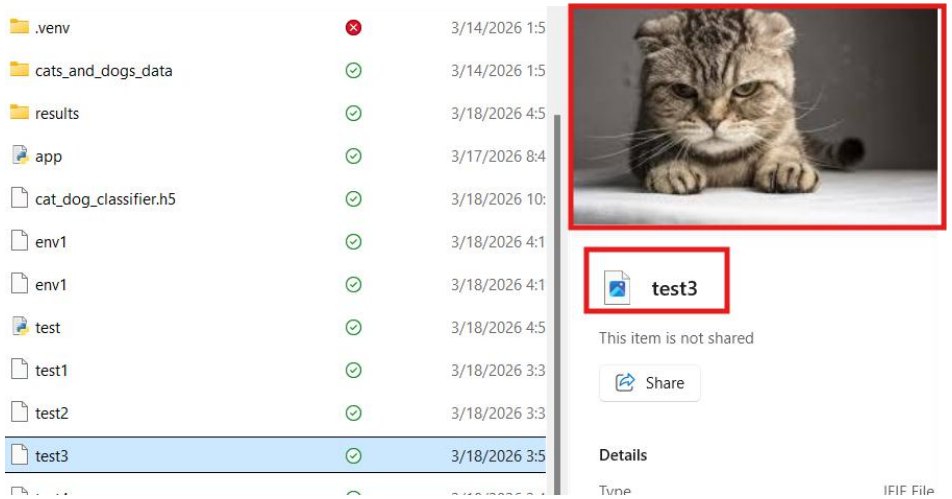
Results





- The model specifically optimizes its parameters (weights) to minimize error on the training set, adapting directly to its peculiarities.
  - During training, the model adjusts its weights to minimize loss on the training set. It essentially fits itself to that data.
  - The validation set is new to the model. It hasn't been used to update weights, so the model won't perform quite as well on it.
  - Models often start to "memorize" patterns (and even noise) in training data.

Results




```
Terminal    Local
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

(.venv) PS C:\Users\Marina\OneDrive\Desktop\Artificial Intelligence\MSP> python test.py test3.jfif
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1773884600.070912   25556 port.cc:153] oneDNN custom operations are on. You may see slightly d
ferent computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1773884632.899254   25556 port.cc:153] oneDNN custom operations are on. You may see slightly d
ferent computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even
TensorFlow-DirectML plugin.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metr
test3.jfif -> Cat (99.99%)

(.venv) PS C:\Users\Marina\OneDrive\Desktop\Artificial Intelligence\MSP>
```

Links To Code and Video

- Github
  - https://github.com/marinarizkk/ai-hw-winter-2026-nn/tree/main/MRP
- YouTube
  - https://youtu.be/Gdfhgzh-f9o

THANK YOU!